

# ME314 Homework 7

## Submission instructions

Deliverables that should be included with your submission are shown in **bold** at the end of each problem statement and the corresponding supplemental material. **Your homework will be graded IFF you submit a single PDF, .mp4 videos of animations when requested and a link to a Google colab file that meet all the requirements outlined below.**

- List the names of students you've collaborated with on this homework assignment.
- Include all of your code (and handwritten solutions when applicable) used to complete the problems.
- Highlight your answers (i.e. **bold** and outline the answers) for handwritten or markdown questions and include simplified code outputs (e.g. .simplify()) for python questions.
- Enable Google Colab permission for viewing
- Click Share in the upper right corner
- Under "Get Link" click "Share with..." or "Change"
- Then make sure it says "Anyone with Link" and "Editor" under the dropdown menu
- Make sure all cells are run before submitting (i.e. check the permission by running your code in a private mode)
- Please don't make changes to your file after submitting, so we can grade it!
- Submit a link to your Google Colab file that has been run (before the submission deadline) and don't edit it afterwards!

**NOTE:** This Jupyter Notebook file serves as a template for you to start homework. Make sure you first copy this template to your own Google driver (click "File" -> "Save a copy in Drive"), and then start to edit it.

```
In [1]: #####
# If you're using Google Colab, uncomment this section by selecting the whole section and press
# ctrl+'/' on your and keyboard. Run it before you start programming, this will enable the nice
# LaTeX "display()" function for you. If you're using the local Jupyter environment, leave it alone
#####
import sympy as sym
import numpy as np
import matplotlib.pyplot as plt

def custom_latex_printer(exp,**options):
    from google.colab.output._publish import javascript
    url = "https://cdnjs.cloudflare.com/ajax/libs/mathjax/3.1.1/latest.js?config=TeX-AMS_HTML"
    javascript(url=url)
    return sym.printing.latex(exp,**options)
sym.init_printing(use_latex="mathjax",latex_printer=custom_latex_printer)
```

## Problem 1 (20pts)

Show that if  $R \in SO(n)$ , then the matrix  $A = \frac{d}{dt}(R)R^{-1}$  is skew symmetric.

**Turn in: A scanned (or photograph from your phone or webcam) copy of your hand written solution. Or you can use \LaTeX.**

$$1) \quad R \in SO(n)$$

$$A = \frac{d}{dt}(R) R^{-1} \quad (\text{or}) \quad A = \frac{d}{dt}(R) R^T$$

We know that  $R \cdot R^T = I = R^T R$

$$\frac{d}{dt}(R R^T) = \frac{d}{dt}(I)$$

$$\Rightarrow \underbrace{\dot{R} R^T}_A + \underbrace{R \cdot \dot{R}^T}_{A^T} = 0$$

$$\Rightarrow A + A^T = 0$$

Thus  $A$  is Skew Symmetric matrix

## Problem 2 (20pts)

Show that

$$\hat{\omega} r_b = -\hat{r}_b \omega,$$

where  $\omega$  and  $r_b$  are both  $3 \times 1$  vectors.

**Turn in: A scanned (or photograph from your phone or webcam) copy of your hand written solution. Or you can use \LaTeX.**

2)

$\hat{\omega} \times r_b$

$\hat{\omega} \times r_b$  is mathematically equivalent to  $\omega \times r_b$

$$a \times b = -b \times a$$

$$\Rightarrow \underline{\omega \times r_b} = - \underline{r_b \times \omega}$$

$$= -\hat{r}_b \omega$$

$$\therefore \hat{\omega} \times r_b = -\hat{r}_b \omega$$

(OR)

$$\hat{\omega} = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix} \quad \text{where } \omega = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} \quad \text{and } r_b = \begin{bmatrix} r_{b1} \\ r_{b2} \\ r_{b3} \end{bmatrix}$$

Computing  $\hat{\omega} \times r_b$

$$\hat{\omega} \times r_b = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix} \begin{bmatrix} r_{b1} \\ r_{b2} \\ r_{b3} \end{bmatrix} = \begin{bmatrix} -\omega_3 r_{b2} + \omega_2 r_{b3} \\ \omega_3 r_{b1} - \omega_1 r_{b3} \\ -\omega_2 r_{b1} + \omega_1 r_{b3} \end{bmatrix} \quad \text{--- ①}$$

$$\hat{r}_b = \begin{bmatrix} 0 & -r_{b3} & r_{b2} \\ r_{b3} & 0 & -r_{b1} \\ -r_{b2} & r_{b1} & 0 \end{bmatrix} \quad \text{where } r_b = \begin{bmatrix} r_{b1} \\ r_{b2} \\ r_{b3} \end{bmatrix} \quad \text{and } \omega = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix}$$

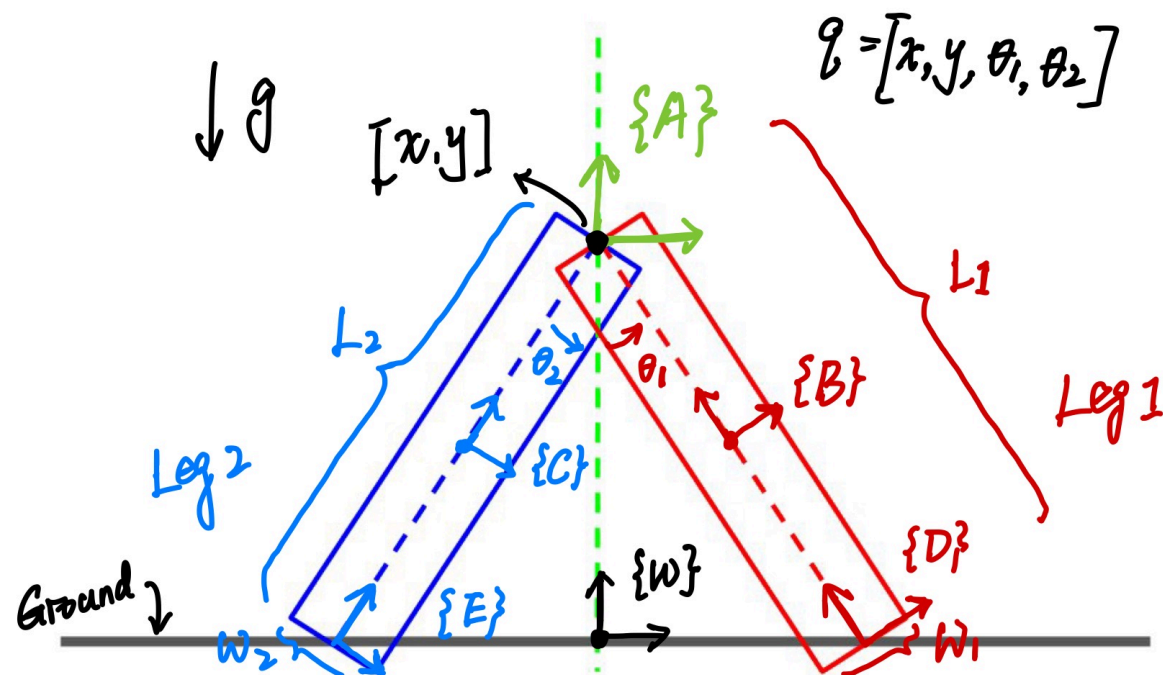
Computing  $\delta_b \omega$

$$\hat{R}_b \omega = \begin{bmatrix} 0 & -R_{b3} & R_{b2} \\ R_{b3} & 0 & -R_{b1} \\ -R_{b2} & R_{b1} & 0 \end{bmatrix} \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} = \begin{bmatrix} -R_{b3} \omega_2 + R_{b2} \omega_3 \\ R_{b3} \omega_1 - R_{b1} \omega_3 \\ -R_{b2} \omega_1 + R_{b1} \omega_3 \end{bmatrix} \quad -\textcircled{2}$$

From  $\textcircled{1}$  and  $\textcircled{2}$

$$\hat{\omega} R_b = -\hat{R}_b \omega$$

```
In [2]: from IPython.core.display import HTML
display(HTML("<table><tr><td><img src='https://github.com/MuchenSun/ME314pngs/raw/master/biped_simplified.jpg' width=600' height='350'></td></tr></table>"))
```



### Problem 3 (60pts)

Consider a person doing the splits (shown in the image above). To simplify the model, we ignore the upper body and assume the knees can not bend --- which means we only need four variables  $q = [x, y, \theta_1, \theta_2]$  to configure the system.  $x$  and  $y$  are the position of the intersection point of the two legs,  $\theta_1$  and  $\theta_2$  are the angles between the legs and the green vertical dash line. The feet are constrained on the ground, and there is no friction between the feet and the ground.

Each leg is a rigid body with length  $L = 1$ , width  $W = 0.2$ , mass  $m = 1$ , and rotational inertia  $J = 1$  (assuming the center of mass is at the center of geometry). Moreover, there are two torques applied on  $\theta_1$  and  $\theta_2$  to control the legs to track a desired trajectory:

$$\theta_1^d(t) = \frac{\pi}{15} + \frac{\pi}{3} \sin^2\left(\frac{t}{2}\right)$$

$$\theta_2^d(t) = -\frac{\pi}{15} - \frac{\pi}{3} \sin^2\left(\frac{t}{2}\right)$$

and the torques are:

$$\begin{aligned}F_{\theta_1} &= -k_1(\theta_1 - \theta_1^d) \\ F_{\theta_2} &= -k_1(\theta_2 - \theta_2^d)\end{aligned}$$

In this problem we use  $k_1 = 20$ .

Given the model description above, define the frames that you need (several example frames are shown in the image as well), simulate the motion of the biped from rest for  $t \in [0, 10]$ ,  $dt = 0.01$ , with initial condition  $q = [0, L_1 \cos(\frac{\pi}{15}), \frac{\pi}{15}, -\frac{\pi}{15}]$ . You will need to modify the animation function to display the leg as a rectangle, an example of the animation can be found at <https://youtu.be/w8XHrEoWTc>.

*Hint 1: Even though this is a 2D system, in order to compute kinetic energy from both translation and rotation you will need to model the system in the 3D world --- the  $z$  coordinate is always zero and the rotation is around the  $z$  axis (based on these facts, what should the  $SE(3)$  matrix and rotational inertia tensor look like?). This also means you need to represent transformations in  $SE(3)$  and the body velocity  $\mathcal{V}_b \in \mathbb{R}^6$ .*

*Hint 2: It could be helpful to define several helper functions for all the matrix operations you will need to use. For example, a function that returns  $SE(3)$  matrices given a rotation angle and 2D translation vector, functions for "hat" and "unhat" operations, a function for the matrix inverse of  $SE(3)$  (which is definitely not the same as the SymPy matrix inverse function), and a function that returns the time derivative of  $SO(3)$  or  $SE(3)$ .*

*Hint 3: In this problem the external force depends on time  $t$ . Therefore, in order to solve for the symbolic solution you need to substitute your configuration variables, which are defined as symbolic functions of time  $t$  (such as  $\theta_1(t)$  and  $\frac{d}{dt}\theta_1(t)$ ), with dummy symbolic variables. For the same reason (the dynamics now explicitly depend on time), you will need to do some tiny modifications on the "integrate" and "simulate" functions, a good reference can be found at [https://en.wikipedia.org/wiki/Runge-Kutta\\_methods](https://en.wikipedia.org/wiki/Runge-Kutta_methods).*

*Hint 4: Symbolically solving this system should be fast, but if you encountered some problem when solving the dynamics symbolically, an alternative method is to solve the system numerically --- substitute in the system state at each time step during simulation and solve for the numerical solution --- but based on my experience, this would cost more than one hour for 500 time steps, so it's not recommended.*

*Hint 5: The animation of this problem is similar to the one in last homework --- the coordinates of the vertices in the body frame are constant, you just need to transfer them back to the world frame using the the transformation matrices you already have in the simulation.*

*Hint 6: Be careful to consider the relationship between the frames and to not build in any implicit assumptions (such as assuming some variables are fixed).*

*Hint 7: The rotation, by convention, is assumed to follow the right hand rule, which means the  $z$ -axis is out of the screen and the positive rotation orientation is counter-clockwise. Make sure you follow a consistent set of positive directions for all the computation.*

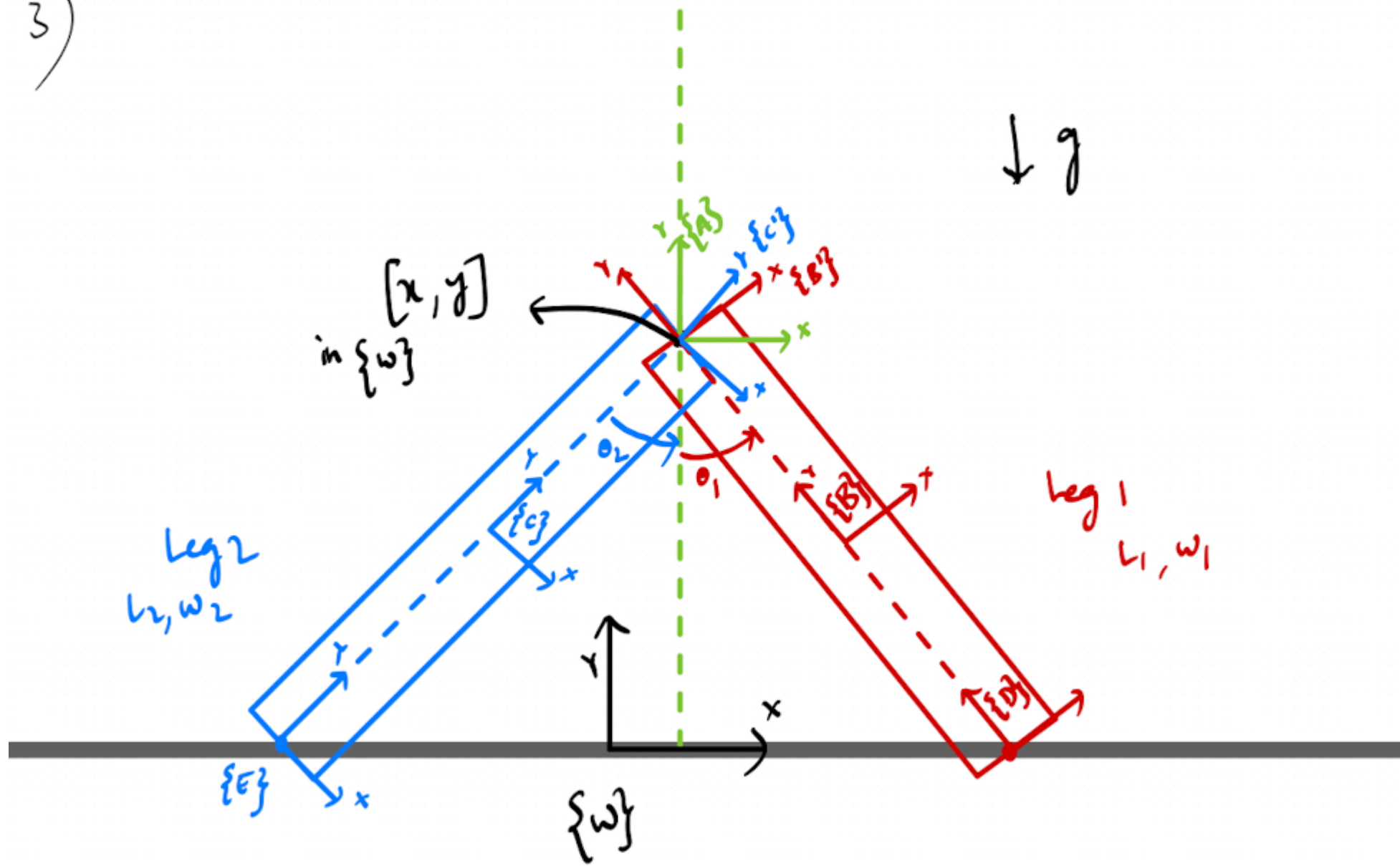
*Hint 8: This problem is designed as a "mini-project", it could help you estimate the complexity of your final project, and you could adjust your proposal based on your experience with this problem.*

**Turn in: A copy of the code used to simulate and animate the system. Also, include a plot of the trajectory and upload a video of the animation separately through Canvas. The video should be in ".mp4" format, you can use screen capture or record the screen directly with your phone.**

In [3]: `## Your code goes here`



3)



$$g_{wA} = g\left(0, \begin{bmatrix} x \\ y \\ 0 \end{bmatrix}\right)$$

leg 1

$$g_{AB'} = g\left(\theta_1, \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}\right) \quad g_{B'B} = g\left(0, \begin{bmatrix} 0 \\ 1/2 \\ 0 \end{bmatrix}\right) \quad g_{BD} = g\left(0, \begin{bmatrix} 0 \\ -1/2 \\ 0 \end{bmatrix}\right)$$

leg 2

$$g_{AC'} = g(-\theta_2, \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}) \quad g_{C'C} = g(0, \begin{bmatrix} 0 \\ -L/2 \\ 0 \end{bmatrix}) \quad g_{CE} = g(0, \begin{bmatrix} 0 \\ L/2 \\ 0 \end{bmatrix})$$

$$g_{WD} = g_{WA} \cdot g_{AB'} \cdot g_{B'B} \cdot g_{BD}$$

$$g_{WE} = g_{WA} \cdot g_{AC'} \cdot g_{C'C} \cdot g_{CE}$$

```
In [4]: # Helper Functions
def theta_p_to_SE3(theta, p):
    """
    Create an SE(3) transformation matrix from a rotation angle and translation vector.
    """
    g = sym.Matrix([
        [sym.cos(theta), -sym.sin(theta), 0.0, p[0]],
        [sym.sin(theta),  sym.cos(theta), 0.0, p[1]],
        [0.0, 0.0, 1.0, 0.0],
        [0.0, 0.0, 0.0, 1.0]
    ])

    return g

def SE3_to_rot_p(g):
    """
    Extract the rotation matrix (R) and translation vector (p) from an SE(3) matrix.
    """
    return g[0:3, 0:3], g[0:3, 3]

def SE3_inv(g):
    """
    Compute the inverse of an SE(3) transformation matrix.
    """
    R, p = SE3_to_rot_p(g)
    R_transpose = R.transpose()
    g_inv = sym.Matrix.vstack(sym.Matrix.hstack(R_transpose, -R_transpose*p), \
                                sym.Matrix([[0.0, 0.0, 0.0, 1.0]]))

    return g_inv

def vec_to_so3(v):
    """
    Convert a 3D angular velocity vector into its skew-symmetric matrix form (so(3)).
    """
    sym
    return sym.Matrix([
        [0, -v[2], v[1]],
        [v[2], 0, -v[0]],
        [-v[1], v[0], 0]
    ])

```



```

def vec_to_se3(v):
    """
    Convert a 6D spatial velocity vector into its se(3) matrix form.
    """
    g_dot = sym.Matrix.vstack( \
        sym.Matrix.hstack(vec_to_so3(sym.Matrix([v[3], v[4], v[5]])), \
            [[v[0]], [v[1]], [v[3]]]), \
        sym.Matrix([[0.0, 0.0, 0.0, 0.0]])
    )
    return g_dot

def se3_to_vec(g_dot):
    """
    Convert an se(3) matrix into its 6D vector representation.
    """
    v = sym.Matrix([g_dot[0, 3], g_dot[1, 3], g_dot[2, 3], g_dot[2, 1], g_dot[0, 2], g_dot[1, 0]])
    return v

def S03_dot_body(R, omega):
    """
    Compute the time derivative of an S0(3) rotation matrix in the body frame.
    """
    omega_hat = vec_to_so3(omega)
    return R*omega_hat

def SE3_dot_body(R, omega, p_dot):
    """
    Compute the time derivative of an SE(3) transformation matrix in the body frame.
    """
    R_dot = S03_dot_body(R, omega)
    g_dot = sym.Matrix.vstack(sym.Matrix.hstack(R_dot, p_dot), \
        sym.Matrix([[0.0, 0.0, 0.0, 0.0]]))

    return g_dot

```

```

In [5]: from sympy.abc import t

# Define the constants
l, w, m, J, k, g = sym.symbols('l, w, m, J, k, g')

# Define the system state theta1, theta2, x and y
theta1 = sym.Function(r'\theta_1')(t)
theta2 = sym.Function(r'\theta_2')(t)
x = sym.Function(r'x')(t)
y = sym.Function(r'y')(t)

# Define the 6x6 mass matrix
# Note that 3 elements of the matrix (m in z direction, J in x and y direction)
# should not have any impact because we are dealing with 2D rigid body motions
mass_matrix = sym.diag(m, m, m, J, J, J)
# display(mass_matrix)

# We are interested in the position of the COM of legs represented in {W} frame
# Compute the homogeneous tranformation matrices
g_wa = theta_p_to_SE3(0, sym.Matrix([x, y, 0]))

# Leg 1
g_abprime = theta_p_to_SE3(theta1, sym.Matrix([0, 0, 0]))
g_bprimeb = theta_p_to_SE3(0, sym.Matrix([0, -l/2, 0]))
g_bd = theta_p_to_SE3(0, sym.Matrix([0, -l/2, 0]))

```

```

g_wb = g_wa * g_abprime * g_bprimeb
g_wd = g_wb * g_bd

print('\033[1mg_wb:')
display(g_wb)
print("/n")

# Leg 2
g_acprime = theta_p_to_SE3(theta2, sym.Matrix([0, 0, 0]))
g_cprimec = theta_p_to_SE3(0, sym.Matrix([0, -l/2, 0]))
g_ce = theta_p_to_SE3(0, sym.Matrix([0, -l/2, 0]))

g_wc = g_wa * g_acprime * g_cprimec
g_we = g_wc * g_ce

print('\033[1mg_wc:')
display(g_wc)

```

**g\_wb:**

$$\begin{bmatrix} \cos(\theta_1(t)) & -\sin(\theta_1(t)) & 0 & \frac{l \sin(\theta_1(t))}{2} + 1.0x(t) \\ \sin(\theta_1(t)) & \cos(\theta_1(t)) & 0 & -\frac{l \cos(\theta_1(t))}{2} + 1.0y(t) \\ 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$$

/n

**g\_wc:**

$$\begin{bmatrix} \cos(\theta_2(t)) & -\sin(\theta_2(t)) & 0 & \frac{l \sin(\theta_2(t))}{2} + 1.0x(t) \\ \sin(\theta_2(t)) & \cos(\theta_2(t)) & 0 & -\frac{l \cos(\theta_2(t))}{2} + 1.0y(t) \\ 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$$

```

In [6]: # Compute body twists of leg 1 and leg 2
V_wb = se3_to_vec(SE3_inv(g_wb) * SE3_dot_body(g_wb[0:3, 0:3], [0, 0, sym.diff(theta1)], g_wb[0:3, 3].diff(t)))
V_wc = se3_to_vec(SE3_inv(g_wc) * SE3_dot_body(g_wc[0:3, 0:3], [0, 0, sym.diff(theta2)], g_wc[0:3, 3].diff(t)))

# display(sym.simplify(V_wb))
# print("\n")
# display(sym.simplify(V_wc))

# Compute KE of leg 1 and leg 2
KE1 = (0.5*(V_wb.T)*mass_matrix*(V_wb))[0]
KE2 = (0.5*(V_wc.T)*mass_matrix*(V_wc))[0]

# display(sym.simplify(KE1))
# display(sym.simplify(KE2))

# Compute the PE of leg 1 and leg 2
PE1 = m*g*(sym.Matrix([[0, 1, 0, 0]]) * g_wb * (sym.Matrix([[0, 0, 0, 1]]).T))[0]
PE2 = m*g*(sym.Matrix([[0, 1, 0, 0]]) * g_wc * (sym.Matrix([[0, 0, 0, 1]]).T))[0]

# display(PE1)
# display(PE2)

KE = KE1 + KE2
PE = PE1 + PE2

```

```
# Compute the lagrangian of the system
lagrangian = KE - PE
lagrangian = sym.simplify(lagrangian)

# Display Lagrangian
print('\033[1mLagrangian of the system: ')
display(lagrangian)
print("\n")\

# Wrap the lagrangian into a SymPy Matrix
L = sym.Matrix([lagrangian])
# display(L)
```

**Lagrangian of the system:**

$$0.5J\left(\frac{d}{dt}\theta_1(t)\right)^2 + 0.5J\left(\frac{d}{dt}\theta_2(t)\right)^2 + 0.5glm \cos(\theta_1(t)) + 0.5glm \cos(\theta_2(t)) - 2.0gmy(t) + 0.125l^2m\left(\frac{d}{dt}\theta_1(t)\right)^2 + 0.125l^2m\left(\frac{d}{dt}\theta_2(t)\right)^2 + 0.5lm \sin(\theta_1(t))\frac{d}{dt}\theta_1(t)\frac{d}{dt}y(t) + 0.5lm \sin(\theta_2(t))\frac{d}{dt}\theta_2(t)\frac{d}{dt}y(t) + 0.5lm \cos(\theta_1(t))\frac{d}{dt}\theta$$

```
In [7]: q = sym.Matrix([x, y, theta1, theta2])
qdot = q.diff(t)
qddot = qdot.diff(t)

# Compute Jacobian
dLdq = L.jacobian(q).T
dLdqdot = L.jacobian(qdot).T

# print('\033[1mEuler Lagrangian Equations')
EL_lhs = dLdqdot.diff(t) - dLdq
display(EL_lhs)
```

$$\begin{bmatrix} -0.5lm \sin(\theta_1(t))\left(\frac{d}{dt}\theta_1(t)\right)^2 - 0.5lm \sin(\theta_2(t))\left(\frac{d}{dt}\theta_2(t)\right)^2 + 0.5lm \cos(\theta_1(t))\frac{d^2}{dt^2}\theta_1(t) + 0.5lm \cos(\theta_2(t))\frac{d^2}{dt^2}\theta_2(t) + 2.0m\frac{d^2}{dt^2}x(t) \\ 2.0gm + 0.5lm \sin(\theta_1(t))\frac{d^2}{dt^2}\theta_1(t) + 0.5lm \sin(\theta_2(t))\frac{d^2}{dt^2}\theta_2(t) + 0.5lm \cos(\theta_1(t))\left(\frac{d}{dt}\theta_1(t)\right)^2 + 0.5lm \cos(\theta_2(t))\left(\frac{d}{dt}\theta_2(t)\right)^2 + 2.0m\frac{d^2}{dt^2}y(t) \\ 1.0J\frac{d^2}{dt^2}\theta_1(t) + 0.5glm \sin(\theta_1(t)) + 0.25l^2m\frac{d^2}{dt^2}\theta_1(t) + 0.5lm \sin(\theta_1(t))\frac{d^2}{dt^2}y(t) + 0.5lm \cos(\theta_1(t))\frac{d^2}{dt^2}x(t) \\ 1.0J\frac{d^2}{dt^2}\theta_2(t) + 0.5glm \sin(\theta_2(t)) + 0.25l^2m\frac{d^2}{dt^2}\theta_2(t) + 0.5lm \sin(\theta_2(t))\frac{d^2}{dt^2}y(t) + 0.5lm \cos(\theta_2(t))\frac{d^2}{dt^2}x(t) \end{bmatrix}$$

```
In [21]: lambda1, lambda2 = sym.symbols(r'\lambda_1, \lambda_2')

# Constraints
# y-coordinate of feet should be equal to zero at all times
phi1 = (sym.Matrix([[0, 1, 0, 0]]) * g_wd * (sym.Matrix([[0, 0, 0, 1]]).T))[0]
phi2 = (sym.Matrix([[0, 1, 0, 0]]) * g_we * (sym.Matrix([[0, 0, 0, 1]]).T))[0]

print('\033[1mConstraints:')
display(phi1)
display(phi2)

phi1 = sym.Matrix([phi1])
phi2 = sym.Matrix([phi2])

# External forces
theta1_d = sym.pi/15 + (sym.sin(t/2)**2)*sym.pi/3
theta2_d = -sym.pi/15 - (sym.sin(t/2)**2)*sym.pi/3

print('\033[1mExternal Forces:')
F_theta1 = -k * (theta1 - theta1_d)
F_theta2 = -k * (theta2 - theta2_d)
```

```
display(F_theta1)
display(F_theta2)

EL_rhs = (lambda1 * phi1.jacobian(q).T) + (lambda2 * phi2.jacobian(q).T) + sym.Matrix([[0, 0, F_theta1, F_theta2]]).T
# display(EL_rhs)
```

#### Constraints:

$$-1.0l \cos(\theta_1(t)) + 1.0y(t)$$

$$-1.0l \cos(\theta_2(t)) + 1.0y(t)$$

#### External Forces:

$$-k \left( \theta_1(t) - \frac{\pi \sin^2\left(\frac{t}{2}\right)}{3} - \frac{\pi}{15} \right)$$

$$-k \left( \theta_2(t) + \frac{\pi \sin^2\left(\frac{t}{2}\right)}{3} + \frac{\pi}{15} \right)$$

In [9]: *# LHS of the extended EL equations*

```
extended_lhs = [
    EL_lhs[0],
    EL_lhs[1],
    EL_lhs[2],
    EL_lhs[3],
    phi1,
    phi1.diff(t),
    phi1.diff(t).diff(t),
    phi2,
    phi2.diff(t),
    phi2.diff(t).diff(t)
]

# RHS of the extended EL equations
extended_rhs = [
    EL_rhs[0],
    EL_rhs[1],
    EL_rhs[2],
    EL_rhs[3],
    0,
    0,
    0,
    0,
    0,
    0
]

EL_eqns = sym.Eq(sym.Matrix(extended_lhs), sym.Matrix(extended_rhs))
# display(EL_eqns)
```

In [10]: *# Solve EL equations*

```
constant_substitutions = {l:1, m:1, J:1, k:20, g:9.8}

# Create dummy variables for easier generation of symbolic solutions
xdot, ydot, theta1_dot, theta2_dot = sym.symbols(r'\dot{x}, \dot{y}, \dot{\theta1}, \dot{\theta2}')
variable_substitutions = {qdot[0]:xdot, qdot[1]:ydot, qdot[2]:theta1_dot, qdot[3]:theta2_dot}
```

```
EL_eqns = EL_eqns.subs(constant_substitutions)
el_solns = sym.solve(EL_eqns, [qddot[0], qddot[1], qddot[2], qddot[3], lambda1, lambda2])
```

```
el_solns[qddot[0]] = el_solns[qddot[0]].subs(variable_substitutions)
el_solns[qddot[1]] = el_solns[qddot[1]].subs(variable_substitutions)
el_solns[qddot[2]] = el_solns[qddot[2]].subs(variable_substitutions)
el_solns[qddot[3]] = el_solns[qddot[3]].subs(variable_substitutions)
el_solns[lambda1] = el_solns[lambda1].subs(variable_substitutions)
el_solns[lambda2] = el_solns[lambda2].subs(variable_substitutions)
```

```
In [11]: xddot_func = sym.lambdify([q[0], q[1], q[2], q[3], xdot, ydot, theta1_dot, theta2_dot, t], el_solns[qddot[0]])
yddot_func = sym.lambdify([q[0], q[1], q[2], q[3], xdot, ydot, theta1_dot, theta2_dot, t], el_solns[qddot[1]])
theta1ddot_func = sym.lambdify([q[0], q[1], q[2], q[3], xdot, ydot, theta1_dot, theta2_dot, t], el_solns[qddot[2]])
theta2ddot_func = sym.lambdify([q[0], q[1], q[2], q[3], xdot, ydot, theta1_dot, theta2_dot, t], el_solns[qddot[3]])
```

```
In [12]: # @title
## Numerical Solution for the system (Please ignore)

# def xddot_func(x_k, y_k, theta1_k, theta2_k, xdot_k, ydot_k, theta1dot_k, theta2dot_k, t_k):
#     subs_ = {
#         q[0]:x_k,
#         q[1]:y_k,
#         q[2]:theta1_k,
#         q[3]:theta2_k,
#         qdot[0]:xdot_k,
#         qdot[1]:ydot_k,
#         qdot[2]:theta1dot_k,
#         qdot[3]:theta2dot_k,
#         t:t_k
#     }

#     return sym.N(el_solns[qddot[0]].subs(subs_))

# def yddot_func(x_k, y_k, theta1_k, theta2_k, xdot_k, ydot_k, theta1dot_k, theta2dot_k, t_k):
#     subs_ = {
#         q[0]:x_k,
#         q[1]:y_k,
#         q[2]:theta1_k,
#         q[3]:theta2_k,
#         qdot[0]:xdot_k,
#         qdot[1]:ydot_k,
#         qdot[2]:theta1dot_k,
#         qdot[3]:theta2dot_k,
#         t:t_k
#     }

#     return sym.N(el_solns[qddot[1]].subs(subs_))

# def theta1ddot_func(x_k, y_k, theta1_k, theta2_k, xdot_k, ydot_k, theta1dot_k, theta2dot_k, t_k):
#     subs_ = {
#         q[0]:x_k,
#         q[1]:y_k,
#         q[2]:theta1_k,
#         q[3]:theta2_k,
#         qdot[0]:xdot_k,
#         qdot[1]:ydot_k,
#         qdot[2]:theta1dot_k,
#         qdot[3]:theta2dot_k,
#         t:t_k
#     }
```

```

#     }

#     return sym.N(el_solns[qddot[2]].subs(subs_))

# def theta2ddot_func(x_k, y_k, theta1_k, theta2_k, xdot_k, ydot_k, thetaldot_k, theta2dot_k, t_k):
#     subs_ = {
#         q[0]:x_k,
#         q[1]:y_k,
#         q[2]:theta1_k,
#         q[3]:theta2_k,
#         qdot[0]:xdot_k,
#         qdot[1]:ydot_k,
#         qdot[2]:thetaldot_k,
#         qdot[3]:theta2dot_k,
#         t:t_k
#     }

#     return sym.N(el_solns[qddot[3]].subs(subs_))

```

```

In [13]: def integrate(f, xt, dt, t):
        """
        Perform numerical integration using the 4th-order Runge-Kutta (RK4) method.

        Parameters:
        =====
        f: Python function
            Derivative of the system ( $\dot{x}(t) = f(x(t), t)$ ).
        xt: NumPy array
            Current state vector  $x(t)$ .
        dt: float
            Time step for integration.
        t: float
            Current simulation time.

        Returns:
        =====
        new_xt: NumPy array
            State vector  $x(t+dt)$  after integration.
        """
        k1 = dt * f(xt, t)
        k2 = dt * f(xt+k1/2., t)
        k3 = dt * f(xt+k2/2., t)
        k4 = dt * f(xt+k3, t)
        new_xt = xt + (1/6.) * (k1+2.0*k2+2.0*k3+k4)
        return new_xt

def simulate(f, x0, tspan, dt, integrate):
    """
    Simulate the trajectory of a dynamical system over a specified time span.

    Parameters:
    =====
    f: Python function
        Derivative of the system ( $\dot{x}(t) = f(x(t), t)$ ).
    x0: NumPy array
        Initial conditions for the system.
    tspan: list
        Time span of simulation [min_time, max_time].
    dt: float
        Time step for numerical integration.
    """

```



```

    integrate: Python function
               Numerical integration method.

    Returns:
    =====
    x_traj: NumPy array
            Simulated trajectory of x(t) with shape (len(x0), num_time_steps).
    """
N = int((max(tspan)-min(tspan))/dt)
x = np.copy(x0)
tvec = np.linspace(min(tspan),max(tspan),N)
xtraj = np.zeros((len(x0),N))
xtraj[:,0] = x0

for i in range(1,N):
    xtraj[:,i]=integrate(f, x, dt, tvec[i])
    x = np.copy(xtraj[:,i])

return xtraj

def biped(x, t):
    """
    Computes the state derivatives for a biped system.

    Parameters:
    =====
    x: NumPy array
        State vector [x, y, theta1, theta2, xdot, ydot, theta1dot, theta2dot].
        - x, y: Position coordinates.
        - theta1, theta2: Angular positions of two legs (or legs).
        - xdot, ydot: Velocities in x and y directions.
        - theta1dot, theta2dot: Angular velocities of the legs.
    t: float
        Current time.

    Returns:
    =====
    A NumPy array of state derivatives:
    [xdot, ydot, theta1dot, theta2dot, xddot, yddot, theta1ddot, theta2ddot].
    """
x_k = x[0]
y_k = x[1]
theta1_k = x[2]
theta2_k = x[3]

xdot_k = x[4]
ydot_k = x[5]
theta1dot_k = x[6]
theta2dot_k = x[7]

t_k = t

xddot_k = xddot_func(x_k, y_k, theta1_k, theta2_k, xdot_k, ydot_k, theta1dot_k, theta2dot_k, t_k)
yddot_k = yddot_func(x_k, y_k, theta1_k, theta2_k, xdot_k, ydot_k, theta1dot_k, theta2dot_k, t_k)
theta1ddot_k = theta1ddot_func(x_k, y_k, theta1_k, theta2_k, xdot_k, ydot_k, theta1dot_k, theta2dot_k, t_k)
theta2ddot_k = theta2ddot_func(x_k, y_k, theta1_k, theta2_k, xdot_k, ydot_k, theta1dot_k, theta2dot_k, t_k)

return np.array([xdot_k, ydot_k, theta1dot_k, theta2dot_k, xddot_k, yddot_k, theta1ddot_k, theta2ddot_k])

```

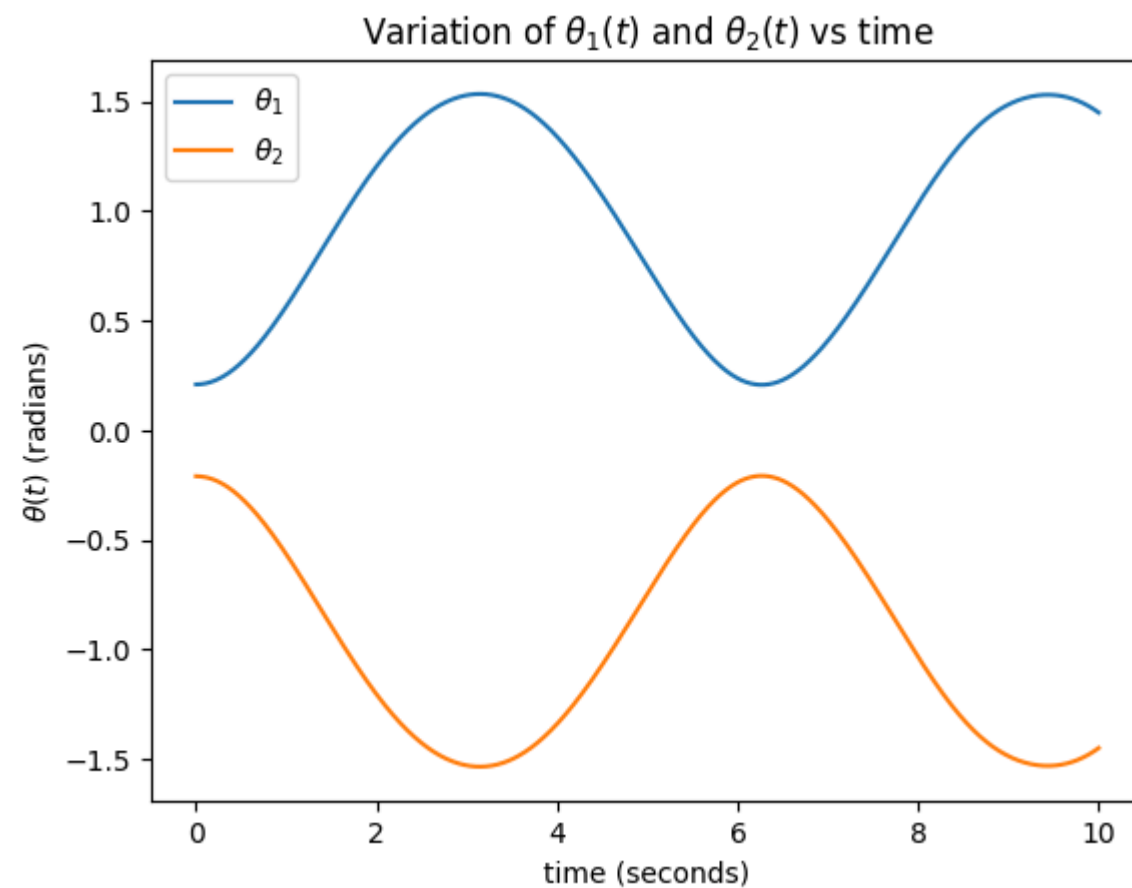


```
In [14]: # Parameters of the simulation
t0 = 0
tf = 10
dt = 0.01
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

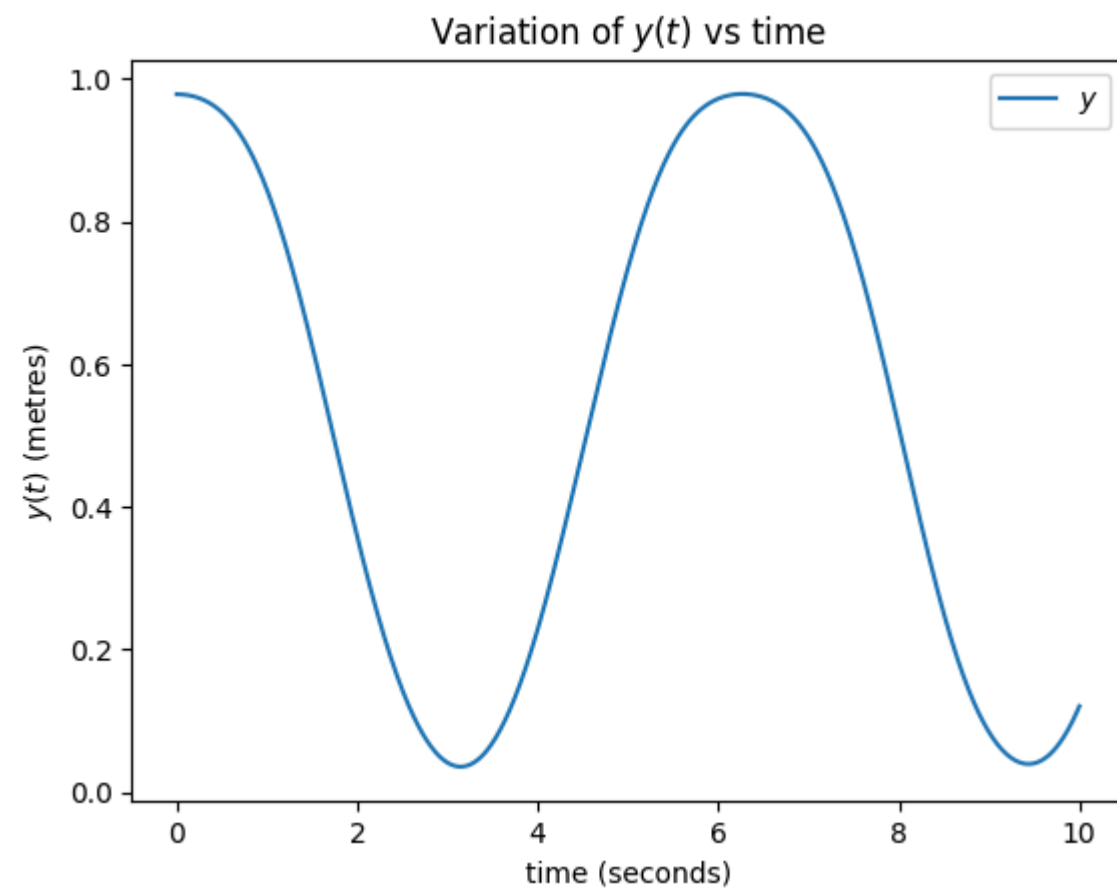
# Initial condition (Considering zero initial velocity)
# State vector [x, y, theta1, theta2, xdot, ydot, thetadot, theta2dot]
x0 = [0, np.cos(np.pi/15), np.pi/15, -np.pi/15, 0, 0, 0, 0]

# Simulate the system
x_traj = simulate(biped, x0, [t0, tf], dt, integrate)
```

```
In [15]: # plot theta1(t) and theta2(t)
plt.plot(tvec, x_traj[2, :], label=r"$\theta_1$")
plt.plot(tvec, x_traj[3, :], label=r"$\theta_2$")
plt.title(r'Variation of $\theta_1(t)$ and $\theta_2(t)$ vs time')
plt.xlabel("time (seconds)")
plt.ylabel(r"$\theta(t)$ (radians)")
plt.legend()
plt.show()
```



```
In [16]: # plot y(t)
plt.plot(tvec, x_traj[1, :], label=r"$y$")
plt.title(r'Variation of $y(t)$ vs time')
plt.xlabel("time (seconds)")
plt.ylabel(r"$y(t)$ (metres)")
plt.legend()
plt.show()
```



```
In [19]: def homogeneous_transform(x):
    return np.concatenate((x, [1]))

def animate_biped(state_vect, L=1, W=1, T=10):
    """
    Function to generate web-based animation of biped

    Parameters:
    =====
    state_vect:
        trajectory of x, y, theta1 and theta2, should be a NumPy array with
        shape of (4,N)
    L:
        length of the biped leg
    W:
        width of the biped leg
    T:
        length/seconds of animation duration

    Returns: None
    """

    #####
    # Imports required for animation.
    from plotly.offline import init_notebook_mode, iplot
    from IPython.display import display, HTML
    import plotly.graph_objects as go

    #####
    # Browser configuration.
    def configure_plotly_browser_state():

```

```

import IPython
display(IPython.core.display.HTML('''
<script src="/static/components/requirejs/require.js"></script>
<script>
  requirejs.config({
    paths: {
      base: '/static/base',
      plotly: 'https://cdn.plot.ly/plotly-1.5.1.min.js?noext',
    },
  });
</script>
'''))

configure_plotly_browser_state()
init_notebook_mode(connected=False)

#####
# Getting data from state vector.
x = state_vect[0, :]
y = state_vect[1, :]
theta1 = state_vect[2, :]
theta2 = state_vect[3, :]

N = len(state_vect[0]) # Need this for specifying length of simulation

leg1 = np.zeros((2, 4, N))
leg2 = np.zeros((2, 4, N))

foot1 = np.zeros((2, N))
foot2 = np.zeros((2, N))

rect_x = np.array([W/2, -W/2, -W/2, W/2])
rect_y = np.array([L/2, L/2, -L/2, -L/2])
rect_z = np.zeros(4)

for i in range(N):
    # Evaluate homogenous transformation
    g_wb = np.array([
        np.cos(theta1[i]), -np.sin(theta1[i]), 0, 0.5*L*np.sin(theta1[i]) + x[i],
        np.sin(theta1[i]), np.cos(theta1[i]), 0, -0.5*L*np.cos(theta1[i]) + y[i],
        0, 0, 1, 0],
        [0, 0, 0, 1]
    ])

    g_wc = np.array([
        np.cos(theta2[i]), -np.sin(theta2[i]), 0, 0.5*L*np.sin(theta2[i]) + x[i],
        np.sin(theta2[i]), np.cos(theta2[i]), 0, -0.5*L*np.cos(theta2[i]) + y[i],
        0, 0, 1, 0],
        [0, 0, 0, 1]
    ])

    for j in range(4):
        leg1[:, j, i] = g_wb.dot(homogeneous_transform([rect_x[j], rect_y[j], rect_z[j]]))[0:2]
        leg2[:, j, i] = g_wc.dot(homogeneous_transform([rect_x[j], rect_y[j], rect_z[j]]))[0:2]

    foot1[:, i] = g_wb.dot(homogeneous_transform([0, -L/2, 0]))[0:2]
    foot2[:, i] = g_wc.dot(homogeneous_transform([0, -L/2, 0]))[0:2]

#####
# Using these to specify axis limits.
xm=np.min(leg2[0, 2, :])-0.2

```

```

xM=np.max(leg1[0, 3, :])+0.2
ym=np.min(y)-0.2
yM=np.max(y)+0.2

#####
# Defining data dictionary.
# Trajectories are here.
data=[
    dict(name="Ground"),
    dict(name='Leg 1'),
    dict(name='Leg 2'),
    dict(name='Foot 1'),
    dict(name='Foot 2'),
]

#####
# Preparing simulation layout.
# Title and axis ranges are here.
layout=dict(xaxis=dict(range=[xm, xM], autorange=False, zeroline=False,dtick=1),
            yaxis=dict(range=[ym, yM], autorange=False, zeroline=False,scaleanchor = "x",dtick=1),
            title='Biped Simulation',
            hovermode='closest',
            updatemenus= [{ 'type': 'buttons',
                            'buttons': [{ 'label': 'Play','method': 'animate',
                                           'args': [None, {'frame': {'duration': T, 'redraw': False}}]},
                                           { 'args': [[None], {'frame': {'duration': T, 'redraw': False}, 'mode': 'immediate',
                                           'transition': {'duration': 0}}], 'label': 'Pause','method': 'animate'}
                                ]
                          }
            ])

#####
# Defining the frames of the simulation.
frames=[dict(data=[
    dict(x=[xm, xM],
        y=[0, 0],
        mode='lines',
        line=dict(color='grey', width=3)
    ),
    dict(x=[leg1[0, 0, k], leg1[0, 1, k], leg1[0, 2, k], leg1[0, 3, k], leg1[0, 0, k]],
        y=[leg1[1, 0, k], leg1[1, 1, k], leg1[1, 2, k], leg1[1, 3, k], leg1[1, 0, k]],
        mode='lines',
        line=dict(color='red', width=3)
    ),
    dict(x=[leg2[0, 0, k], leg2[0, 1, k], leg2[0, 2, k], leg2[0, 3, k], leg2[0, 0, k]],
        y=[leg2[1, 0, k], leg2[1, 1, k], leg2[1, 2, k], leg2[1, 3, k], leg2[1, 0, k]],
        mode='lines',
        line=dict(color='blue', width=3)
    ),
    go.Scatter(
        x=[foot1[0, k]],
        y=[foot1[1, k]],
        mode="markers",
        marker=dict(color="black", size=7)),
    go.Scatter(
        x=[foot2[0, k]],
        y=[foot2[1, k]],
        mode="markers",
        marker=dict(color="black", size=7)),

```

```
    ]) for k in range(N)]

#####
# Putting it all together and plotting.
figure1=dict(data=data, layout=layout, frames=frames)
iplot(figure1)
```

```
In [20]: animate_biped(x_traj, L=1, W=0.2, T=10)
```

Biped Simulation

Play

Pause

